

AWS Certified AI Practitioner

AIF-C01 — Detailed Domain Notes

A complete, self-built study set. Every concept is presented in three layers — **What it is**, **Why it works** (causal mechanism), and **Applied** (a realistic scenario) — with the official domain code, a difficulty tier, and a confidence tag. High-stakes concepts add a **Fails when** line (the invalidation condition). Depth is weighted to the official blueprint, heaviest domain first.

Domain	Weight	Order
D3 — Applications of Foundation Models	28%	1st (heaviest)
D2 — Fundamentals of Generative AI	24%	2nd
D1 — Fundamentals of AI and ML	20%	3rd
D4 — Guidelines for Responsible AI	14%	4th
D5 — Security, Compliance & Governance	14%	5th

Exam logistics: 65 questions (50 scored + 15 unscored), 700/1000 scaled to pass, ~90 minutes, Pearson VUE (center or online-proctored), valid 3 years. Foundational level.

How to use these notes

This set does not just define services — it teaches the **decision logic the exam tests**. AIF-C01 is overwhelmingly a scenario exam: it hands you a situation with a decisive constraint and asks which service, technique, or control fits. Reading definitions is not enough; you must map constraints to answers.

The elimination method (use on every question)

Step	What to do
1. Read the LAST sentence first	The decisive constraint lives there: “least operational overhead,” “no ML team,” “most recent information,” “never traverse the public internet.”
2. Classify the task	Service selection? Concept? Technique? Trade-off? Security/responsible-AI control?
3. Eliminate categorically-wrong options	Cross off anything in the wrong service family (e.g. an image service for a text task).
4. Break the tie on the constraint	Between survivors, which one uniquely satisfies the constraint keyword?
5. Check the distractor pairs	AWS pairs look-alikes on purpose. Know the boundary (RAG vs fine-tune, Bedrock vs SageMaker, etc.).

Tag legend: **Dxx** = official domain, **Recall / Application / Analysis** = difficulty tier (matches how deeply the exam probes), **HIGH / MODERATE** = confidence. **[VERSION-SENSITIVE]** flags a fact worth re-verifying against current AWS docs.

Domain 3 — Applications of Foundation Models (28%)

The heaviest domain and the exam's center of gravity. It is where the managed→custom spectrum, prompt engineering, RAG vs fine-tuning, Amazon Bedrock, and the managed AI services all live. If your study time is limited, spend the most of it here.

Amazon Bedrock — serverless foundation-model access

D3 Application HIGH

What it is. A fully managed, serverless service that exposes foundation models from multiple providers (Amazon Titan, Anthropic Claude, Meta Llama, Mistral, Cohere, AI21, Stability) through one unified API.

Why it works. AWS owns the compute, scaling, and endpoints, so a team consumes models as an API call with no infrastructure to provision. That is the causal reason Bedrock is the default answer whenever a scenario says “no ML expertise,” “without managing infrastructure,” or “fully managed.”

Applied. A travel company wants pre-trained generative image capability with no ML team and no desire to host models → Amazon Bedrock, not JumpStart (which still hands you an endpoint to manage).

Fails when. It is the wrong pick when the constraint is “deploy inside our VPC” or “open-source model we control” — that pushes you to SageMaker JumpStart.

Bedrock Knowledge Bases — managed RAG

D3 Application HIGH

What it is. A managed Retrieval-Augmented Generation pipeline: it ingests your documents, embeds and stores them in a vector store, retrieves the most relevant chunks per query, and feeds them to an FM to generate a grounded answer.

Why it works. The FM answers from retrieved company data at inference time instead of from frozen training weights, so the answer reflects current, private information and hallucination drops. That mechanism is why “supplement the model with our private/proprietary data” maps to a Knowledge Base.

Applied. A company using Titan on Bedrock must answer from internal policy documents that change monthly → create a Bedrock Knowledge Base (RAG). Lowering temperature or switching models does not add data access.

Fails when. It underperforms fine-tuning when the need is a consistent house style or deeply specialized domain reasoning rather than fresh facts.

Bedrock Guardrails — runtime content safety

D3 / D4 Application HIGH

What it is. A policy layer that filters generative inputs and outputs: denied topics, content filters (hate, violence, sexual, misconduct), PII redaction, word filters, and prompt-attack detection.

Why it works. It evaluates text against your configured policies at inference time and blocks or masks violations before they reach the user — a content control, distinct from access control. That is why “filter,” “block topics,” or “detect prompt attacks” = Guardrails.

Applied. A chatbot must refuse to discuss competitors and never emit customer SSNs → configure Guardrails with a denied topic and PII redaction.

Fails when. It does not address model bias or explainability — that is SageMaker Clarify. Confusing the two is the classic trap.

Bedrock Agents — multi-step orchestration

D3 **Application** **HIGH**

What it is. An orchestration layer that lets an FM break a request into steps, call APIs/Lambda functions and knowledge sources, and chain the results into a completed task.

Why it works. The model plans, invokes tools, and consolidates outputs, so it automates repetitive multi-step workflows rather than answering a single prompt. In multi-agent setups a supervisor agent decomposes the task, delegates to specialized sub-agents, and merges their outputs.

Applied. A retailer wants customer-support automation that looks up orders and processes returns across systems → Bedrock Agents orchestrating the API calls.

Fails when. Overkill for a single-turn Q&A; that a plain prompt or a Knowledge Base already satisfies.

Bedrock pricing — on-demand vs provisioned throughput

D3 **Recall** **MODERATE**

What it is. On-demand bills per input/output token with no commitment; provisioned throughput reserves dedicated model capacity for a term in exchange for guaranteed throughput.

Why it works. On-demand scales freely and costs nothing when idle, so it fits limited budgets and flexible/spiky usage; provisioned suits steady high-volume production needing guaranteed capacity.

Applied. “Limited budget, flexibility, no long-term commitment” → on-demand.

Fails when. On-demand can be costlier and rate-limited at sustained high volume — then provisioned wins.

The managed→custom spectrum & SageMaker build options

D3 / D1 **Analysis** **HIGH**

What it is. A continuum of AWS options ordered by effort/control: Managed AI services → Bedrock → SageMaker JumpStart (pre-built models you deploy, incl. in your VPC) → Canvas (no-code) / Autopilot (AutoML with visibility) → SageMaker Studio (full custom) → train from scratch.

Why it works. Each step trades less effort for more control. The scenario's constraints select the point: team ML skill, need to own infrastructure/VPC, customization depth, and budget.

Applied. “No code, business analyst” → Canvas. “Automatically build with full visibility” → Autopilot. “Pre-built model deployed in our VPC / limited expertise” → JumpStart. “Full custom build-train-deploy” → Studio.

Fails when. Picking a heavier tier than the constraints require loses “least operational overhead” questions; picking a lighter tier fails “full control / deploy in VPC” questions.

Prompt engineering spectrum: zero-shot → few-shot → fine-tune → continued pre-training

D3 **Analysis** **HIGH**

What it is. Techniques ordered by effort: zero-shot (instruction only), few-shot (a handful of in-prompt examples), fine-tuning (retrain on labeled prompt→response pairs), continued pre-training (extend base knowledge on a large unlabeled corpus).

Why it works. Adding examples or training data increases control and consistency at rising cost. The exam maps constraint keywords to points on this line.

Applied. “Least operational effort” + needs style guidance → few-shot. “No examples, generic task” → zero-shot. “Specialize / consistent tone / domain expert” → fine-tune. “Completely different domain” → continued pre-training.

Fails when. Few-shot cannot inject large external knowledge (context-window limited) — for that use RAG.

RAG vs fine-tuning — the #1 decision

D3 **Analysis** **HIGH**

What it is. RAG retrieves external facts at inference and augments the prompt; fine-tuning adjusts model weights during training to bake in behavior.

Why it works. RAG changes the *facts the model sees now* without retraining; fine-tuning changes the model's learned *behavior/style* permanently. Therefore changing/fresh facts → RAG; consistent style or specialized domain reasoning → fine-tuning.

Applied. “Answer from the most recent information” / “least development effort to add external knowledge” → RAG. “Specialize the model in legal/medical language and tone” → fine-tuning.

Fails when. RAG is wrong when data is static and you need consistent voice; fine-tuning is wrong when facts change often (you'd retrain constantly) or you must cite live sources.

Managed AI services — keyword → service

D3 **Recall** **HIGH**

These purpose-built services need no ML knowledge. On the exam a single keyword picks the service:

If the scenario needs...	Service
Sentiment / entities / NLP on text	Amazon Comprehend
Objects, faces, moderation in images/video	Amazon Rekognition
Text/keys/tables from scanned docs & forms (OCR)	Amazon Textract
Text-to-speech (natural voice)	Amazon Polly
Speech-to-text (with custom vocab)	Amazon Transcribe
Translation across languages	Amazon Translate
Chatbots / conversational IVR	Amazon Lex
Enterprise search that returns documents	Amazon Kendra
Personalized recommendations	Amazon Personalize
Contact-center platform	Amazon Connect
Medical NLP on clinical text	Amazon Comprehend Medical
Health data at scale (FHIR)	AWS HealthLake

Amazon Q — Developer vs Business

D3 **Recall** **MODERATE**

What it is. Two generative assistants: Q Developer works in the IDE/coding context (code generation, review, MRs); Q Business answers questions over enterprise knowledge/documents.

Why it works. Each is grounded in a different corpus — code vs company knowledge — so the context in the stem picks the answer.

Applied. “Help engineers write and review code” → Q Developer. “Let employees ask questions of internal docs” → Q Business.

Inference options & latency (SageMaker)

D3 / D1

Application

HIGH

What it is. Real-time endpoint (persistent, ~sub-60s), Asynchronous (queued, up to ~1 hour, large payloads), Batch Transform (offline, hours-to-days), and Serverless Inference (auto-scales to zero, no infra to manage).

Why it works. The deployment mode matches the latency/throughput and payload profile of the workload — a persistent warm endpoint for live traffic, a queue for big or bursty jobs, offline batch for bulk scoring.

Applied. “Low latency / real-time” → real-time. “Large payload, up to an hour, queue” → async. “Offline, no urgency, days” → batch. “Serve without managing infrastructure” → serverless inference.

Vector stores & OpenSearch (why RAG retrieval works)

D3 / D2

Application

MODERATE

What it is. Embeddings (e.g. Titan Embeddings) turn text into dense vectors; a vector store such as Amazon OpenSearch indexes them and does nearest-neighbor (k-NN) similarity search.

Why it works. Semantically similar text lands near in vector space, so nearest-neighbor search surfaces the passages most relevant to a query — the retrieval step that grounds RAG.

Applied. “Which OpenSearch capability enables a vector-database/RAG pattern?” → scalable index management + k-NN search.

Domain 2 — Fundamentals of Generative AI (24%)

The concepts that make generative AI work: foundation models, embeddings, transformers, GANs, diffusion, context windows, hallucination, and the metrics used to evaluate generated output.

Foundation models (FMs)

D2 Recall HIGH

What it is. Large, general-purpose models pre-trained on broad, diverse data that can be adapted to many downstream tasks via prompting, RAG, or fine-tuning.

Why it works. Scale + broad pre-training produce general capability, so one model serves many tasks instead of training a narrow model per task — the economic premise of generative AI on Bedrock.

Applied. Titan, Claude, Llama used as-is (zero-shot), then prompted, RAG-augmented, or fine-tuned as needs grow.

Amazon Titan

D2 Recall MODERATE

What it is. AWS's first-party FM family on Bedrock: Titan Text, Titan Embeddings, Titan Image Generator.

Why it works. Being first-party, Titan is the default AWS-native option and the source of embeddings used by Bedrock Knowledge Bases.

Applied. "Which are AWS's own foundation models?" → the Titan family (other Bedrock models are third-party: Anthropic, Meta, Mistral, Cohere, AI21, Stability).

Vector embeddings

D2 Recall HIGH

What it is. Numeric representations of text (or images) as dense vectors in a high-dimensional space where distance encodes semantic similarity.

Why it works. Meaning becomes geometry, so similarity search, clustering, and retrieval reduce to distance math — the foundation of semantic search and RAG.

Applied. Embeddings → vector DB (OpenSearch) → nearest-neighbor retrieval → Bedrock Knowledge Base answer.

Context window

D2 Application HIGH

What it is. The maximum number of tokens (input + output) a model can consider in a single request.

Why it works. Everything the model reasons over must fit in this budget, so it caps how much prompt, retrieved context, and answer you can include — critical for RAG, where retrieved chunks must fit.

Applied. "How much information can we put into one prompt when choosing an FM?" → the context window.

Transformers & LLMs

D2 Recall MODERATE

What it is. The neural architecture behind modern LLMs; self-attention weighs how strongly each token relates to every other token in the sequence.

Why it works. Attention captures long-range context in parallel (unlike older sequential RNNs), which is why transformers scale to large models and long contexts and power text generation like GPT-style models.

Applied. “Generate SQL from natural language” → a generative (GPT-style transformer) model, not ResNet/SVM/WaveNet.

GANs (generator vs discriminator)

D2

Application

MODERATE

What it is. A Generative Adversarial Network pairs a generator (creates fakes) against a discriminator (classifies real vs fake); they train in competition.

Why it works. The generator improves by trying to fool the discriminator while the discriminator improves at detecting fakes; the adversarial pressure drives both toward realistic output.

Applied. “Role of the discriminator?” → classify data as real or fake. “Basic idea?” → generator fools while discriminator detects.

Diffusion models (Stable Diffusion)

D2

Recall

MODERATE

What it is. Text-to-image models that generate images by iteratively denoising random noise conditioned on a text prompt (Stability AI's Stable Diffusion, available on Bedrock).

Why it works. Learning to reverse a noising process lets the model synthesize a coherent image that matches the prompt.

Applied. “Primary function of Stable Diffusion?” → generate images from text descriptions.

Hallucination

D2 / D4

Recall

HIGH

What it is. When a model generates fabricated or factually incorrect content stated confidently.

Why it works. The model predicts plausible next tokens, not verified truth, so with no grounding it can invent facts — the central reliability risk of generative AI.

Applied. Mitigations the exam rewards: RAG (ground in real data), Guardrails (filter outputs), human review (A2I), and lower temperature.

Evaluation metrics — match the metric to the task

D2

Analysis

HIGH

What it is. Text generation/summarization: ROUGE (n-gram overlap with a reference), BERTScore (semantic similarity via embeddings). Classification: Accuracy, F1 (precision/recall balance). Regression: RMSE, R^2 .

Why it works. Each metric measures the error type its task produces — overlap for summaries, semantic closeness for meaning, correct-count for classification, error magnitude for numeric prediction. Using the wrong family measures nothing.

Applied. “Evaluate a summarizer” → ROUGE / BERTScore. “How many images classified correctly?” → Accuracy. “Predicting a price” → RMSE / R^2 .

Fails when. ROUGE on a classifier or Accuracy on a regressor is a category error — a common distractor.

Customization trade-offs

D2

Analysis

MODERATE

What it is. Customizing/fine-tuning an FM on proprietary data raises cost and implementation complexity (and adds data/ops work) in exchange for better task fit.

Why it works. Training compute, data preparation, and engineering effort are the real prices of control — accuracy and relevance improve, they do not degrade.

Applied. “Trade-offs of customizing your own FM?” (pick two) → higher cost + higher implementation complexity.

Domain 1 — Fundamentals of AI and Machine Learning (20%)

The ML vocabulary every other domain reuses: learning types, task types, training pitfalls, hyperparameters, and the SageMaker build/prep tooling.

Supervised vs unsupervised vs reinforcement learning

D1 Recall HIGH

What it is. Supervised learns from labeled data; unsupervised finds structure in unlabeled data; reinforcement learns a policy by maximizing cumulative reward through trial-and-error with an environment.

Why it works. The presence/absence of labels and the feedback signal determine the paradigm, which in turn dictates the algorithms and services that apply.

Applied. Spam vs not-spam / price prediction → supervised. Customer segmentation → unsupervised (clustering). Robot learns to navigate a maze → reinforcement.

Fails when. Calling clustering “supervised” or an RL objective “minimize loss” (that’s supervised optimization) are the planted errors.

Classification vs regression

D1 Application HIGH

What it is. Two supervised task types: classification predicts a category/label; regression predicts a continuous number.

Why it works. The output type decides the task and therefore the metric family: labels → Accuracy/F1, numbers → RMSE/R².

Applied. Spam/not-spam, disease category → classification. House price, temperature, sales total → regression.

Overfitting vs underfitting

D1 Application HIGH

What it is. Overfitting: high training accuracy but poor performance on unseen data (memorized noise). Underfitting: a model too simple to capture the pattern (poor on both).

Why it works. A model too complex for the data fits noise and fails to generalize; too simple and it cannot represent the signal. Fixes for overfitting: more data, regularization, early stopping, cross-validation.

Applied. “High training accuracy, low test accuracy” → overfitting (the exam often describes the symptom without the word). “Simple model, complex pattern” → underfitting.

Hyperparameters vs parameters

D1 Recall HIGH

What it is. Hyperparameters are set before training (learning rate, batch size, epochs, #layers); parameters (weights, biases) are learned during training.

Why it works. You configure the learning process (hyperparameters); the process produces the model’s internals (parameters). That boundary is exactly what the exam probes.

Applied. “Which is a hyperparameter?” → learning rate. “Increase accuracy toward an acceptance level” → increase epochs (more passes over the data).

Fails when. Temperature is an inference control, not a training hyperparameter — a favorite distractor.

Improving a model: feature engineering & hyperparameter tuning

D1

Application

MODERATE

What it is. Feature engineering adds/derives input variables; hyperparameter tuning searches training configurations to improve performance.

Why it works. More informative features give the model better signal; better hyperparameters change how it learns — two distinct levers on accuracy.

Applied. “Increase the variables in the data AND modify algorithm behavior” (pick two) → feature engineering + hyperparameter tuning.

Data augmentation & dimensionality reduction

D1

Recall

MODERATE

What it is. Augmentation synthetically expands training data via transformations (rotate/flip/crop images); dimensionality reduction (PCA, t-SNE) cuts feature count while preserving information.

Why it works. Augmentation adds variety without new collection (also rebalances classes and curbs overfitting); reduction fights the curse of dimensionality and speeds/stabilizes training.

Applied. “Artificially increase training-set size” → augmentation. “Reduce features while keeping information” → dimensionality reduction.

SageMaker build & prep tooling

D1 / D3

Application

HIGH

What it is. Data Wrangler (prep/preprocess), Ground Truth (labeling with human + ML), Feature Store (reusable features), Canvas (no-code), Autopilot (AutoML with visibility), JumpStart (pre-built models), Studio (full IDE), Model Monitor (drift), Clarify (bias/explainability), Model Cards (documentation), Pipelines (workflow).

Why it works. Each tool owns one stage of the ML lifecycle; the exam gives a stage keyword and expects the matching tool.

Applied. “No code / business analyst” → Canvas. “Automatically build with visibility” → Autopilot. “Label data” → Ground Truth. “Reuse features across models” → Feature Store. “Detect drift in production” → Model Monitor.

Inference vs training (and AI agents)

D1

Recall

MODERATE

What it is. Training = learning from data; inference = using the trained model on new inputs; deployment = making it available. An AI agent perceives (percepts), plans, and acts on its environment to maximize a utility/goal.

Why it works. Once deployed, every prediction on new data is inference — a distinct phase from the one-time (or periodic) training. Agent types range from reflex to model-based, goal-based, utility-based, and learning agents.

Applied. “Deployed model analyzes a new image” → inference. “Two types of intelligent agents” (reflex, model-based reflex) → AI agents.

Domain 4 — Guidelines for Responsible AI (14%)

Bias, fairness, explainability, human oversight, and the AWS tools that implement them. The exam pairs a problem with the right responsible-AI control.

SageMaker Clarify — bias & explainability

D4 **Application** **HIGH**

What it is. Detects bias in data and models (pre- and post-training) and explains predictions via feature attribution (SHAP values).

Why it works. It runs statistical bias analysis and attribution on training data and predictions, producing transparency reports — the control for “bias,” “fairness,” “explainable,” “transparent.”

Applied. A regulated medical model must be transparent and explainable → SageMaker Clarify.

Fails when. It is not a content filter (that's Guardrails) and not production drift monitoring (that's Model Monitor).

Guardrails vs Clarify (the responsible-AI pair)

D4 / D3 **Analysis** **HIGH**

What it is. Guardrails filters generative content at runtime (inputs/outputs); Clarify analyzes ML bias and explainability at training/eval time.

Why it works. They operate at different layers and solve different problems: content safety vs statistical fairness/transparentcy. Naming the layer resolves the question.

Applied. “Block harmful topics / redact PII in a chatbot” → Guardrails. “Measure and explain model bias” → Clarify.

Bias in data → mitigation

D4 **Application** **HIGH**

What it is. When training data is imbalanced/biased, the model inherits the bias.

Why it works. Bias enters at the source, so the fix is at the source: rebalance with data augmentation for under-represented classes, add fairness metrics to evaluation, and correct the training data.

Applied. “Input data for an image generator is biased” → data augmentation for imbalanced classes. “Deploy an LLM responsibly” (pick two) → fairness metrics + mitigate bias in training data.

Human oversight — Amazon A2I

D4 / D3 **Application** **HIGH**

What it is. Amazon Augmented AI routes low-confidence predictions to human reviewers.

Why it works. A confidence threshold decides when a human must confirm, keeping people in the loop for uncertain or high-stakes decisions — the “human review / not confident / safety net” answer.

Applied. Document-processing predictions below a confidence threshold → A2I human review.

Fails when. Unlike Guardrails (automatic block), A2I inserts a human — different safety mechanisms.

SageMaker Model Cards & AI ethics principles

D4 **Recall** **MODERATE**

What it is. Model Cards are immutable records of a model's intended uses, risk ratings, training details, and evaluation results. Responsible-AI principles include human-centricity, preserving human agency (autonomy),

universal design, and continual monitoring; sustainability spans social, economic, and environmental aims.

Why it works. Documented, immutable model records support audit, compliance, and transparency; ethics principles keep humans in control and systems accountable.

Applied. “Record intended uses and risk ratings for governance” → Model Cards. “Autonomy principle” → preserve human agency (human-centric AI).

Generative AI limitations & moderation

D4

Application

MODERATE

What it is. Generative models can produce biased or inappropriate content and require human review; content-moderation APIs / Guardrails screen outputs.

Why it works. Because output is probabilistic and ungrounded by default, oversight and moderation are necessary controls.

Applied. “Limitation to be aware of when generating product descriptions” → may produce biased/inappropriate content needing review. “Ensure a chatbot doesn't return inappropriate images” → moderation APIs / Guardrails.

Domain 5 — Security, Compliance & Governance for AI Solutions (14%)

Access control, logging/audit, vulnerability and data protection, compliance evidence, and the shared-responsibility model applied to generative AI. The exam routes a security verb to a service.

AWS IAM — access control

D5 Application HIGH

What it is. Identity and Access Management governs who can do what to which resources via roles and least-privilege policies.

Why it works. IAM authenticates and authorizes every API action, so any “secure access / authorize / permissions / least privilege” question resolves to IAM — including securing access to Amazon Bedrock.

Applied. “Secure access to Bedrock” → IAM roles with least privilege (paired with clear prompts for safe LLM use).

CloudTrail vs CloudWatch (the observability pair)

D5 Application HIGH

What it is. CloudTrail logs API calls (who did what, when); CloudWatch monitors metrics, alarms, and dashboards (is it healthy?).

Why it works. Trail answers accountability/audit questions; Watch answers operational-health questions. The verb in the stem picks the service.

Applied. “Record API calls to Bedrock / identify unauthorized access attempts” → CloudTrail. “Monitor metrics and alarms” → CloudWatch. (End-to-end request tracing → X-Ray.)

Inspector vs Macie (the protection pair)

D5 Application HIGH

What it is. Amazon Inspector scans infrastructure/code for vulnerabilities (EC2, ECR, Lambda); Amazon Macie discovers sensitive data / PII in S3.

Why it works. Inspector works at the infrastructure layer (CVEs), Macie at the data layer (content classification) — different targets, different questions.

Applied. “Assess security posture / find vulnerabilities in EC2 & ECR” → Inspector. “Find PII/SSNs/credit cards in S3” → Macie.

Compliance evidence — AWS Artifact

D5 Recall MODERATE

What it is. A self-service portal for AWS compliance reports and agreements (SOC, ISO, PCI attestations).

Why it works. It provides the third-party audit artifacts customers need for their own compliance — the “compliance reports / certifications” answer.

Applied. “Obtain/receive notice of ISV compliance reports” → AWS Artifact. (Audit Manager collects evidence; Trusted Advisor gives best-practice checks; Config tracks resource configuration.)

Generative-AI shared responsibility (security scoping)

D5 Analysis HIGH

What it is. Security ownership scales with customization: using a third-party app with embedded GenAI (least) → building on a third-party FM API → fine-tuning an FM with your data → building/training a model from scratch (most).

Why it works. The more of the stack you control, the more security responsibility shifts to you — the shared-responsibility model applied to GenAI.

Applied. “Which scope gives the MOST security ownership?” → building and training a model from scratch on your own data.

Data protection & prompt-injection defense

D5

Application

MODERATE

What it is. Protect data with encryption (KMS), network isolation (VPC/PrivateLink), and IAM; defend LLMs from prompt-injection with system prompts/templates that teach refusal, plus Guardrails prompt-attack detection.

Why it works. Layered controls keep data private and instructions trustworthy; a template that recognizes manipulation patterns hardens the model at the application layer.

Applied. “Prevent an LLM being manipulated by prompt attacks” → a prompt template that detects attack patterns (and Guardrails). “Demonstrate regulatory compliance” (pick two) → threat detection + data protection.

Fails when. Raising temperature or trimming tokens does nothing for injection — planted wrong answers.

Exam-day paired-distractor cheat sheet

Nine pairs account for a large share of missed questions. Learn the boundary, not just the names.

Pair	Pick the first when...	Pick the second when...
RAG vs Fine-tuning	facts change / “most recent” / least effort to add data	stable domain / consistent tone / “specialize”
Bedrock vs SageMaker JumpStart	serverless, no ML team, no infra	deploy in VPC / open-source model you own
Guardrails vs Clarify	filter content / block topics / prompt attacks	bias / fairness / explainability
Kendra vs Bedrock KB	return documents (enterprise search)	generate an answer from documents (RAG)
Q Developer vs Q Business	code / IDE / merge requests	enterprise knowledge Q&A;
CloudTrail vs CloudWatch	API logging / who-did-what-when	metrics / alarms / health
Autopilot vs Canvas	“automatically” + full visibility	no code / business analyst
Inspector vs Macie	vulnerabilities in EC2/ECR	sensitive data / PII in S3
Temperature vs Top-p	scales the whole distribution (randomness)	truncates to nucleus (cumulative p)